COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	07/24/26	**DATE SUBMITTED:**	07/24/26
NAME:	Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Quillbot, Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Since I'm not so good at constructing words, I tend to rely on grammar checkers to easily explain my answers.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Output for Single Linked List:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\mjean\Documents\dsahoa3p1> & C:\Users\mjean\Documents\dsahoa3p1\mainhare.exe
Testing the ListTraversal Function
C->P->E->0->1->0
Testing the sllInsertHead Function
G->C->P->E->0->1->0
Testing the sllGeneralInsert Function
G->C->P->E->E->0->1->0
Testing the sllDeleteNode Function (Delete C)
G->P->E->E->0->1->0
Testing the sllDeleteNode Function (Delete P)
G->E->E->0->1->0
Testing the sllDeleteList Function
```

T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

mainhare.cpp mainhare.exe

mainhare.cpp > main()

```cpp
#include <iostream>
#include "singly_ll.h"

int main()
{
    Singlylist<char>* head = nullptr;
    Singlylist<char>* second = nullptr;
    Singlylist<char>* third = nullptr;
    Singlylist<char>* fourth = nullptr;
    Singlylist<char>* fifth = nullptr;
    Singlylist<char>* last = nullptr;

    // Allocate memory
    head = new Singlylist<char>;
    second = new Singlylist<char>;
    third = new Singlylist<char>;
    fourth = new Singlylist<char>;
    fifth = new Singlylist<char>;
    last = new Singlylist<char>;

    // Create CPE010
    head->data = 'C';
    second->data = 'P';
    third->data = 'E';
    fourth->data = '0';
    fifth->data = '1';
    last->data = '0';

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    fifth->next = last;
    last->next = nullptr;
```

```
    // Traverse
    std::cout << "Testing the ListTraversal Function" << std::endl;
    ListTraversal(head);

    // Insert Head
    std::cout << "Testing the sllInsertHead Function" << std::endl;
    sllInsertHead('G', &head);
    ListTraversal(head);

    // General Insert (after P)
    std::cout << "Testing the sllGeneralInsert Function" << std::endl;
    sllGeneralInsert('E', head->next->next);
    ListTraversal(head);

    // Delete C
    std::cout << "Testing the sllDeleteNode Function (Delete C)" << std::endl;
    sllDeleteNode('C', &head);
    ListTraversal(head);

    // Delete P
    std::cout << "Testing the sllDeleteNode Function (Delete P)" << std::endl;
    sllDeleteNode('P', &head);
    ListTraversal(head);

    // Delete List
    std::cout << "Testing the sllDeleteList Function" << std::endl;
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}
```

singly_ll.h > ...

```h
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

#include <iostream>

template <typename T>
class Singlylist {
public:
    T data;
    Singlylist<T>* next;
};

template <typename T>
void ListTraversal(Singlylist<T>* node) {

    while (node != nullptr) {

        std::cout << node->data;

        if (node->next != nullptr) {
            std::cout << "->";
        }

        node = node->next;
    }

    std::cout << std::endl;
}

template <typename T>
void sllInsertHead(T newData, Singlylist<T>** currentHead) {

    Singlylist<T>* newNode = new Singlylist<T>;
    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;
}
```

singly_ll.h > ...

```
1   #ifndef SINGLY_LL_H
39  template <typename T>
40  void sllGeneralInsert(T newData, Singlylist<T>* prevNode) {
41
42      if (prevNode == nullptr) {
43          std::cout << "Previous node cannot be null" << std::endl;
44          return;
45      }
46
47      Singlylist<T>* newNode = new Singlylist<T>;
48      newNode->data = newData;
49
50      newNode->next = prevNode->next;
51      prevNode->next = newNode;
52  }
53
54  template <typename T>
55  void sllInsertEnd(T newData, Singlylist<T>** head) {
56
57      Singlylist<T>* newNode = new Singlylist<T>;
58      newNode->data = newData;
59      newNode->next = nullptr;
60
61      if (*head == nullptr) {
62          *head = newNode;
63          return;
64      }
65
66      Singlylist<T>* currentNode = *head;
67
68      while (currentNode->next != nullptr) {
69          currentNode = currentNode->next;
70      }
71
72      currentNode->next = newNode;
73  }
74
```

```
74
75  template <typename T>
76  void sllDeleteNode(T findData, Singlylist<T>** head) {
77
78      if (*head == nullptr) {
79          return;
80      }
81
82      Singlylist<T>* currNode = *head;
83      Singlylist<T>* prevNode = nullptr;
84
85      while (currNode != nullptr && currNode->data != findData) {
86          prevNode = currNode;
87          currNode = currNode->next;
88      }
89
90      if (currNode == nullptr) {
91          return;
92      }
93
94      if (prevNode == nullptr) {
95          *head = currNode->next;
96      } else {
97          prevNode->next = currNode->next;
98      }
99
100     delete currNode;
101 }
102
103 template <typename T>
104 void sllDeleteList(Singlylist<T>** head) {
105
106     Singlylist<T>* current = *head;
```

```
85      while (currNode != nullptr && currNode->data != findData) {
86          prevNode = currNode;
87          currNode = currNode->next;
88      }
89
90      if (currNode == nullptr) {
91          return;
92      }
93
94      if (prevNode == nullptr) {
95          *head = currNode->next;
96      } else {
97          prevNode->next = currNode->next;
98      }
99
100     delete currNode;
101 }
102
103 template <typename T>
104 void sllDeleteList(Singlylist<T>** head) {
105
106     Singlylist<T>* current = *head;
107
108     while (current != nullptr) {
109         Singlylist<T>* temp = current;
110         current = current->next;
111         delete temp;
112     }
113
114     *head = nullptr;
115 }
116
117 #endif
```

Output for Doubly Linked List:

Traversal

```
std::cout << "Testing the DllTraverse Function" << std::endl;
dllTraverse(head);
```

Console Output

```
PS C:\Users\mjean\Documents\hare_hoa3p1> & C:\Users\mjean\Documents\hare_hoa3p1\mainhare-1.exe
Testing the DllTraverse Function
Forward: C->P->E
Reverse: E->P->C
```

Insert Head

```
    // Insert at head
    std::cout << "Testing the DllInsertHead Function" << std::endl;
    dllInsertHead('X', &head);
    dllTraverse(head);
```

Console Output

```
Testing the DllInsertHead Function
Forward: X->C->P->E
Reverse: E->P->C->X
```

Insert End

```
// Insert at end
std::cout << "Testing the DllInsertEnd Function" << std::endl;
dllInsertEnd('O', head);
dllTraverse(head);
```

Console Output

```
Testing the DllInsertEnd Function
Forward: X->C->P->E->O
Reverse: O->E->P->C->X
```

General Insert

```
// General insert
std::cout << "Testing the DllGeneralInsert Function" << std::endl;
dllGeneralInsert('E', head);
dllTraverse(head);
```

Console Output

```
Testing the DllGeneralInsert Function
Forward: X->E->C->P->E->O
Reverse: O->E->P->C->X
```

Delete Node

```
// Delete node
std::cout << "Testing the DllDeleteNode Function" << std::endl;
dllDeleteNode('C', &head);
dllTraverse(head);


std::cout << "Testing the DllDeleteNode Function" << std::endl;
dllDeleteNode('P', &head);
dllTraverse(head);
```

Console Output

```
Testing the DllDeleteNode Function
Forward: X->E->P->E->O
Reverse: O->E->P->C->X

Testing the DllDeleteNode Function
Forward: X->E->E->O
Reverse: O->E->P->C->X
```

Delete List

```
// Delete entire list
std::cout << "Testing the DllDeleteList Function" << std::endl;
dllDeleteList(&head);
dllTraverse(head);
```

Console Output

```
Testing the DllDeleteList Function
The List is empty.
```

My Analysis:

The doubly linked list was made by modifying the singly linked list and adding a **prev** pointer in every node. Because of this, the list can be traversed in both forward and backward directions. The program was able to perform traversal, insertion, and deletion correctly while keeping the links between the nodes. Instead of using the same letters from the module example, I used different characters to make my own test case. Even if the data is different, the result is still the same because the linked list operations work properly regardless of the characters stored in each node.

COMPUTER ENGINEERING

QUEZON CITY

Supplementary Activity:

mainsupplehare.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "suppleplaylist.h"

int main()
{
    Playlist<std::string>* head = nullptr;

    // Create Playlist
    insertSong(std::string("Pusong Bato"), &head);
    insertSong(std::string("Digi Digi Dong"), &head);
    insertSong(std::string("Hawak mo ang beat"), &head);

    std::cout << "Testing the PlayAllSongs Function" << std::endl;
    playAllSongs(head);

    // Insert Song
    std::cout << "Testing the InsertSong Function" << std::endl;
    insertSong(std::string("Daddy's Home"), &head);
    playAllSongs(head);

    // Current song
    Playlist<std::string>* currentSong = head;

    // Next Song
    std::cout << "Testing the NextSong Function" << std::endl;
    nextSong(&currentSong);
    std::cout << "Current Song: " << currentSong->song << std::endl;

    // Previous Song
    std::cout << "Testing the PreviousSong Function" << std::endl;
    previousSong(head, &currentSong);
    std::cout << "Current Song: " << currentSong->song << std::endl;
```

```
35      // Remove Song
36      std::cout << "Testing the RemoveSong Function" << std::endl;
37      removeSong(std::string("Digi Digi Dong"), &head);
38      playAllSongs(head);
39
40      // Play All Songs
41      std::cout << "Testing the PlayAllSongs Function" << std::endl;
42      playAllSongs(head);
43
44      return 0;
45  }
```

T.I.P TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

suppleplaylist.h > removeSong<T>(T, Playlist<T>**)

```h
#ifndef SUPPLEPLAYLIST_H
#define SUPPLEPLAYLIST_H

#include <iostream>
#include <string>

template <typename T>
class Playlist
{
public:
    T song;
    Playlist<T>* next;
};

// Create a new song node
template <typename T>
Playlist<T>* createSong(T newSong)
{
    Playlist<T>* newNode = new Playlist<T>;
    newNode->song = newSong;
    newNode->next = newNode;

    return newNode;
}
```

```cpp
// Insert song at the end
template <typename T>
void insertSong(T newSong, Playlist<T>** head)
{
    Playlist<T>* newNode = createSong(newSong);

    if (*head == nullptr)
    {
        *head = newNode;
        return;
    }

    Playlist<T>* current = *head;

    while (current->next != *head)
    {
        current = current->next;
    }

    current->next = newNode;
    newNode->next = *head;
}
```

```cpp
// Remove a song
template <typename T>
void removeSong(T findSong, Playlist<T>** head)
{
    if (*head == nullptr)
    {
        return;
    }

    Playlist<T>* current = *head;
    Playlist<T>* previous = nullptr;

    // Delete first node
    if (current->song == findSong)
    {
        if (current->next == *head)
        {
            delete current;
            *head = nullptr;
            return;
        }

        Playlist<T>* last = *head;

        while (last->next != *head)
        {
            last = last->next;
        }
```

```
78          last->next = current->next;
79          *head = current->next;
80
81          delete current;
82          return;
83      }
84
85      previous = current;
86      current = current->next;
87
88      while (current != *head)
89      {
90          if (current->song == findSong)
91          {
92              previous->next = current->next;
93              delete current;
94              return;
95          }
96
97          previous = current;
98          current = current->next;
99      }
100 }
```

```cpp
// Display playlist
template <typename T>
void playAllSongs(Playlist<T>* head)
{
    if (head == nullptr)
    {
        std::cout << "Playlist is empty." << std::endl;
        return;
    }

    Playlist<T>* current = head;

    do
    {
        std::cout << current->song << std::endl;
        current = current->next;

    } while (current != head);
}

// Next song
template <typename T>
void nextSong(Playlist<T>** currentSong)
{
    if (*currentSong != nullptr)
    {
        *currentSong = (*currentSong)->next;
    }
}
```

```cpp
// Previous song
template <typename T>
void previousSong(Playlist<T>* head, Playlist<T>** currentSong)
{
    if (head == nullptr || *currentSong == nullptr)
    {
        return;
    }

    Playlist<T>* temp = head;

    while (temp->next != *currentSong)
    {
        temp = temp->next;
    }

    *currentSong = temp;
}

#endif
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\mjean\Documents\supple_hoa3p1hare> & C:\Users\mjean\Documents\supple_hoa3p1hare\mainsupplehare.exe
Testing the PlayAllSongs Function
Pusong Bato
Digi Digi Dong
Hawak mo ang beat
Testing the InsertSong Function
Pusong Bato
Digi Digi Dong
Hawak mo ang beat
Daddy's Home
Testing the NextSong Function
Current Song: Digi Digi Dong
Testing the PreviousSong Function
Current Song: Pusong Bato
Testing the RemoveSong Function
Pusong Bato
Hawak mo ang beat
Daddy's Home
Testing the PlayAllSongs Function
Pusong Bato
Hawak mo ang beat
Daddy's Home
PS C:\Users\mjean\Documents\supple_hoa3p1hare>
```

T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.